

Musterprüfung zur Vorbereitung: C-Programmierung

Hinweis: Diese Musterprüfung dient der Vorbereitung auf die Teilprüfung. Sie hat denselben Aufbau, denselben Umfang und denselben Schwierigkeitsgrad, besteht aber aus anderen Aufgaben.

Empfehlung: unter realen Bedingungen lösen – 60 Minuten, ohne Hilfsmittel.

Hinweise:

- **Bearbeitungszeit:** 60 Minuten
- **Keine elektronischen Hilfsmittel (Handy, Laptop, Smartwatch usw.) erlaubt**
- **Nur handschriftliche Unterlagen erlaubt**
- **Bewertung:**
 - 2 Punkte: Antwort korrekt und vollständig
 - 1 Punkt: Teilrichtige Antwort (z.B. nur eine von zwei geforderten Nennungen, Ausgabe korrekt aber Begründung fehlt)
 - 0 Punkte: Falsche oder keine Antwort
- **Gesamtpunkte:** 40
- **Notenskala:** Erreichte Punktzahl x 5 / max. Punktzahl + 1 = Note (auf 1/10 gerundet)

Aufgaben

1. Kontrollfluss if/else

```
int y = 12;
if (y < 5)
    printf("X");
else if (y < 10)
    printf("Y");
else
    printf("Z");
```

Aufgabe: Welche Ausgabe erscheint? Begründen Sie.

2. while-Schleife ergänzen

```
int i = 0;
while ( ____ ) {
    printf("%d ", i);
    ____ ;
}
```

Aufgabe: Vervollständigen Sie die while-Schleife so, dass sie die Zahlen **0, 2, 4, 6, 8** ausgibt.

3. for-Schleife

```
for (int i = 10; i > 0; i -= 3) {
    printf("%d ", i);
}
```

Aufgabe: Welche Ausgabe erscheint?

4. Sprungbefehle (Schleife)

```
int i = 0;
while (i < 10)
{
    i++;
    if (i == 4)
        break;
    printf("%d ", i);
}
```

Aufgabe: Welche Zahlen werden ausgegeben?

5. Datentyp-Konvertierung

```
int a = 9;
int b = 4;
double ergebnis1 = a / b;
double ergebnis2 = (double)a / b;

printf("%.2f %.2f", ergebnis1, ergebnis2);
```

Aufgabe: Welche Ausgabe erscheint? Begründen Sie den Unterschied.

6. Gültigkeitsbereich von Variablen

```
void zaehle() {
    static int a = 0;    // ?
    int b = 0;           // ?
    a++;
    b++;
    printf("%d %d\n", a, b);
}

int main() {
    zaehle();
    zaehle();
    zaehle();
}
```

Aufgabe: Welche Ausgabe erscheint? Begründen Sie den Unterschied zwischen **a** und **b**.

7. Stringzugriff

```
char wort[] = "Motor";  
printf("%c", wort[3]);
```

Aufgabe: Welche Ausgabe erscheint?

8. Array befüllen

```
int werte[5];  
for (int i = 0; i < 5; i++) {  
    werte[i] = ____;  
}
```

Aufgabe: Ergänzen Sie den Code, sodass das Array `werte` die Werte **10, 20, 30, 40, 50** speichert.

9. char-Array

```
char text[8] = "Hi";
```

Aufgabe: Welche Zeichen (inklusive `\0`) stehen tatsächlich im Array `text`?

10. Pointer verändern

```
int zahl = 7;
int *p = &zahl;
*p = *p * 3;
printf("%d", zahl);
```

Aufgabe: Welche Ausgabe erscheint? Begründen Sie.

11. Adressoperator und Dereferenzierung

```
int wert = 20;
int *zeiger = &wert;
```

Aufgabe: Erklären Sie, was die Operatoren `&` und `*` in diesem Zusammenhang bewirken.

12. struct-Zugriff

```
struct Artikel { int nr; double preis; };
struct Artikel a1 = {101, 4.50};
printf("%.2f", a1.preis);
```

Aufgabe: Welche Ausgabe erscheint?

13. Aufzählungstyp enum

```
enum Farbe { ROT = 1, GRUEN, BLAU };  
enum Farbe f = BLAU;  
printf("%d", f);
```

Aufgabe: Welche Ausgabe erscheint? Erklären Sie, wie die Werte vergeben werden.

14. Funktionsaufruf mit Pointer

```
void verdopple(int *wert)  
{  
    *wert = *wert * 2;  
}  
  
int main()  
{  
    int a = 5;  
    verdopple(&a);  
    printf("%d\n", a);  
    return 0;  
}
```

Aufgabe: Welche Ausgabe erscheint? Begründen Sie, warum sich `a` ändert.

15. Funktion

```
int hoch3(int n) { return n*n*n; }  
int main() { printf("%d", hoch3(3)); }
```

Aufgabe: Welche Ausgabe erscheint?

16. Rückgabewerte von Funktionen

Aufgabe: Erklären Sie den Unterschied zwischen einer Funktion mit Rückgabotyp `void` und einer Funktion mit Rückgabotyp `int`. Nennen Sie je ein Anwendungsbeispiel.

17. Modularisierung

Aufgabe: a) Warum teilt man ein grösseres C-Programm in mehrere Dateien auf? Nennen Sie zwei Gründe. b) Warum bindet man mit `#include` immer die `.h`-Datei ein und **nicht** die `.c`-Datei?

18. Rekursion

```
int summe(int n) {  
    if (n == 0) return 0;  
    return n + summe(n-1);  
}
```

Aufgabe: Welchen Wert liefert `summe(5)`?

19. Basisfall bei Rekursion

Aufgabe: Was ist ein Basisfall (Abbruchbedingung) bei einer rekursiven Funktion und was passiert, wenn er fehlt?

20. Funktion schreiben

```
int zaehleGroesser(int werte[], int anzahl, int grenze) {  
  
  
  
  
  
  
}
```

Aufgabe: Vervollständigen Sie die Funktion so, dass sie zurückgibt, **wie viele** Werte im Array grösser als **grenze** sind.

Beispiel: Bei {12, 5, 30, 8, 22} und **grenze = 10** ist das Ergebnis **3**.
